Multiagent Game Data Capture Spec

Version 3.1 - 08/16/2026

1. Proposed session data structure..... 1
2. Player-side capture..... 2
    3.1 player.json..... 2
    3.2 frames.bin..... 2
    3.3 packets.bin &packets.txt..... 2
3. Server-side capture..... 3
    3.1 Server state..... 3
    3.2 Server events..... 3
    3.3 packets.bin &packets.txt..... 3
4. Synchronization model and structure..... 3
5. Game list target..... 3

This document describes the draft of Capture Spec v3, focusing on multiplayer gameplay data. It extends Capture Spec v2 to sessions where N players play the same match on a controlled dedicated server. The per-player capture format remains intentionally close to v2 so downstream ingestion is unchanged; everything multiplayer-specific is introduced as new layers around it: session grouping, server-side capture, and cross-player synchronization. Per-player deliverables follow v2 conventions unless explicitly overridden here.

1. Proposed session data structure

The core unit changes from a single recording to a gameplay session: one match played by N players (4 to 10) on a dedicated server we control. All participants share one session_id.

```JSON
<vendor>/<mm-dd-yyyy>/<game>/<session-id>/
session.json                  # session-level metadata (match-wide)
schema.bin.gz                 # schema to decode binary packets

server/
      server_state.bin        # per-tick authoritative game state
```

```
      events.jsonl      # round/match boundaries, kills, objectives
      packets.bin             # binary packets from server
      packets.txt             # human readable packets

players/
      <player-id>/
            player.json       # per-player metadata
            frames.csv        # v2 schema
            frames.bin        # binary representation of frames.csv
            video.mp4         # POV recording
            packets.bin       # binary packets from client
            packets.txt       # human readable packets
            events.jsonl# same as server side, used for syncing
```

These files and values are still prone to change, but this structure should reflect the structure of data we aim to deliver.

Task boundaries are defined by the round, not fixed durations: recording start/stop is triggered by server round events. A session is deliverable only if it passes sync validation and at least K of N player streams are individually valid.

If a player dies in a round and cannot respawn, we stop the recording to not pollute the dataset with “spectator pov” recordings.

2. Player-side capture

Each expert's machine records exactly as in v2: POV video (1080p/60 target, 720p/30 floor), frames.csv with the full v2 column set (C2W camera matrix **where extractable**, per-frame intrinsics, input_keys, input_actions, mouse deltas), and v2 metadata including input_mouse_convention.

3.1 player.json

Contains metadata for the player (name, ping, ID)

3.2 frames.bin

We propose including a binary version of frames.csv which saves floating point variables with full precision.

3.3 packets.bin &packets.txt

These are the binary representation of all network calls, and their decoded version in text format.

3. Server-side capture

The server is authoritative, so ground-truth game state comes from the server rather than per-client extraction:

3.1 Server state

per-tick entities state. Every player's position, orientation, health, active weapon, alive/dead status, plus relevant world entities (alongside many other fields).

We will include instructions on how to extract this data from its binary format.

3.2 Server events

Semantic events with UTC timestamps. round start and end, kills (attacker, victim, weapon), objectives, player connect/disconnect. These events will vary per game, depending on game type and events available.

These are meant as sync points and will only contain high level events related to the pace of the game.

3.3 packets.bin &packets.txt

The same as the player's networking packets, but from the server's perspective

4. Synchronization model and structure

Three layers:

1. Baseline: OS clock via NTP on all machines (players + server).
2. Measured offset: at session start the capture client pings the server and records clock offset + RTT into player.json, correcting residual drift per player.
3. server events validate alignment post-hoc across all streams.

QA gate: a session passes sync validation if all player streams align within tolerance. Tolerance would heavily depend on both the netcode of the game and the ping of the player. We will need more prototyping to decide what a good Tolerance is.

5. Game list target

We plan to use games with the following conditions:

- Must be self-hostable
- Is open source, or has an open source engine to decode netcode